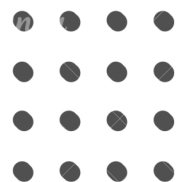


VMES Grinds





643. Maximum Average Subarray I



643. Maximum Average Subarray I

Solved 

Easy

 Topics

 Companies

You are given an integer array `nums` consisting of `n` elements, and an integer `k`.

Find a contiguous subarray whose **length is equal to** `k` that has the maximum average value and return *this value*. Any answer with a calculation error less than 10^{-5} will be accepted.



Example 1:

Input: `nums = [1,12,-5,-6,50,3], k = 4`

Output: `12.75000`

Explanation: Maximum average is $(12 - 5 - 6 + 50) / 4 = 51 / 4 = 12.75$

Example 2:

Input: `nums = [5], k = 1`

Output: `5.00000`



Constraints:

- `n == nums.length`
- `1 <= k <= n <= 105`
- `-104 <= nums[i] <= 104`



Initial Approach

Brute Force

- **For every possible starting position i , calculate the sum of the next k elements from scratch.**



```
class Solution:
    def findMaxAverage(self, nums: List[int], k: int) → float:
        max_sum = float('-inf')
        for i in range(len(nums) - k + 1):
            current_sum = sum(nums[i:i + k])
            max_sum = max(max_sum, current_sum)

        return max_sum / k
```



Optimized Approach



Why slow?

1. $k-1$ elements overlap between adjacent windows.
2. Re-calculating sum of shared elements wastes time.

Sliding Window

1. Subtract the element leaving the window (left).
2. Add the element entering the window (right)



```
class Solution:
    def findMaxAverage(self, nums: List[int], k: int) → float:
        window_sum = sum(nums[:k])
        max_sum = window_sum

        for i in range(k, len(nums)):
            window_sum += nums[i] - nums[i - k]
            max_sum = max(max_sum, window_sum)

        return max_sum / k
```



Time and Space Complexity Analysis



Space - $O(1)$

Time - $O(n*k)$

```
class Solution:
    def findMaxAverage(self, nums: List[int], k: int) -> float:
        max_sum = float('-inf')
        for i in range(len(nums) - k + 1):
            current_sum = sum(nums[i:i + k])
            max_sum = max(max_sum, current_sum)

        return max_sum / k
```

$O(n)$

$O(k)$



Space - $O(1)$

```
class Solution:
    def findMaxAverage(self, nums: List[int], k: int) → float:
        window_sum = sum(nums[:k])
        max_sum = window_sum

        for i in range(k, len(nums)):
            window_sum += nums[i] - nums[i - k]
            max_sum = max(max_sum, window_sum)

        return max_sum / k
```

← Time - $O(n)$



Key Takeaway

VMES Grinds MMM
Chee sar



- Avoid unnecessary recalculations, try to reuse it
- Sliding windows (expand right, shrink left) are almost always $O(N)$.